Monocular Depth Estimation Implementation Progress Report

2026-10-18 | Phases 1-3 | Hardware-adapted implementation

Current result: a working RGB-only depth-perception prototype with real driving-video replay, relative depth maps, colour overlays, saved outputs, portable input verification and repeatable performance profiling.

The project addresses the absence of explicit depth information in ordinary camera images. A pretrained neural network estimates a dense relative-depth map from each RGB frame, providing a perception research prototype that runs on the available laptop hardware.

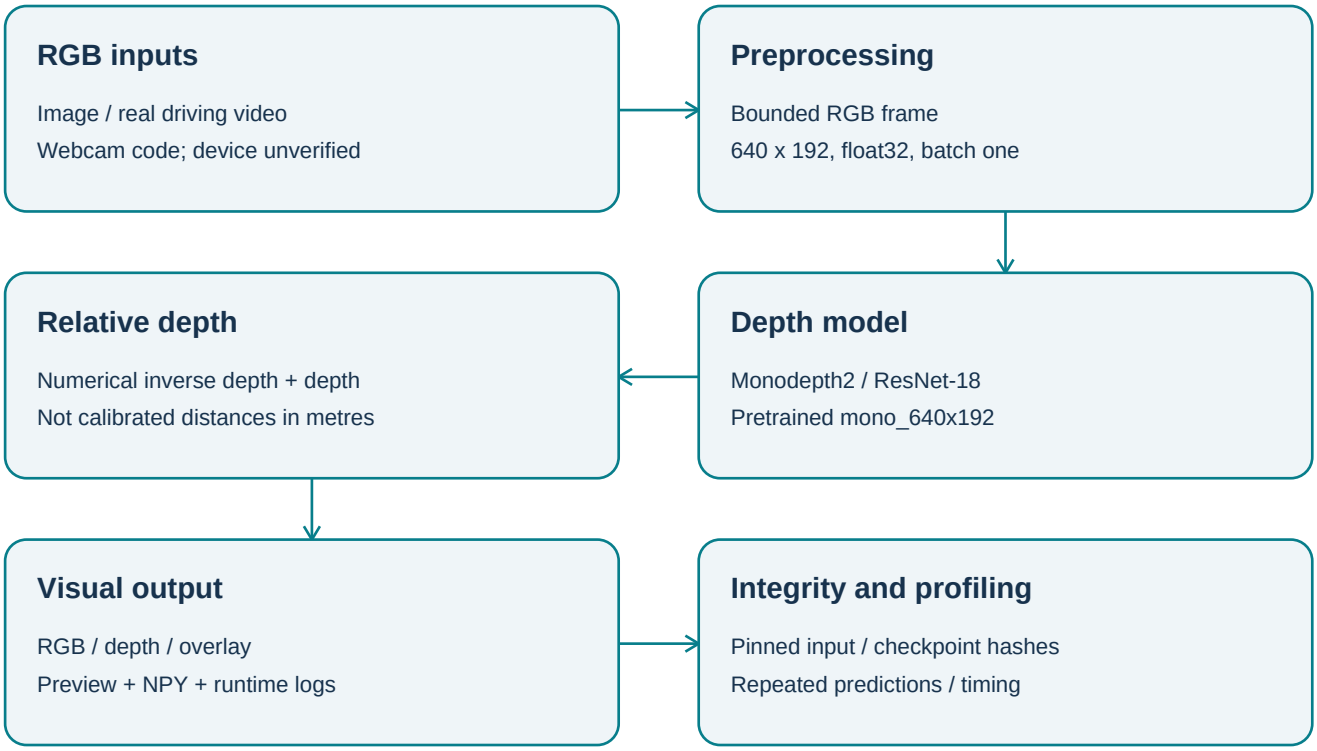
Phase	Implementation position
1 - Foundation	Image/video inference, RGB preprocessing, model loading and shared visualization implemented and tested.
2 - Demo inputs	Real driving sample, retained license, image fallback, portable manifest and checksum verification implemented.
3 - Stability / performance	Warmup-aware repeated profiling, numerical stability gate and measured rendering optimization implemented locally.
4 - Accuracy evaluation	Metric formulas exist; full ground-truth dataset benchmarks remain pending.
5 - Delivery	Setup and transfer instructions exist; final academic submission and presentation-machine acceptance remain pending.

Scope: the latest hardware-adaptation instruction relaxes the original PDF implementation constraints. This report records implemented and tested behavior, not completion of every original PDF deliverable.

Model weights remain pretrained. No project-specific training, validated native CARLA camera run, calibrated metre-scale prediction or autonomous control is claimed.

Source baseline: commit a6dfbc5; Phase 3 is the current working-tree update at report generation. Generated data and runtime evidence remain local.

1. Architecture and model



The webcam and optional CARLA interfaces feed the same preprocessing and inference path, but their real hardware integrations remain unverified. The verified inputs are a saved RGB image and real driving-video replay.

Component	Implemented contract
Encoder	ResNet-18; encoder.pth; strict loading of official checkpoint parameters.
Decoder	Monodepth2 multi-scale depth decoder; depth.pth; scale-zero output used for inference.
Checkpoint	mono_640x192; KITTI-trained monocular baseline; immutable recorded hashes.
Input	RGB HWC uint8 becomes a 1 x 3 x 192 x 640 float32 tensor in [0,1]. Original encoder normalization is retained.
Output	Positive finite inverse-depth and relative-depth maps, 192x640. Depth is the reciprocal of inverse depth.
Execution	Evaluation mode and torch.inference_mode; batch one; automatic CUDA/CPU selection.

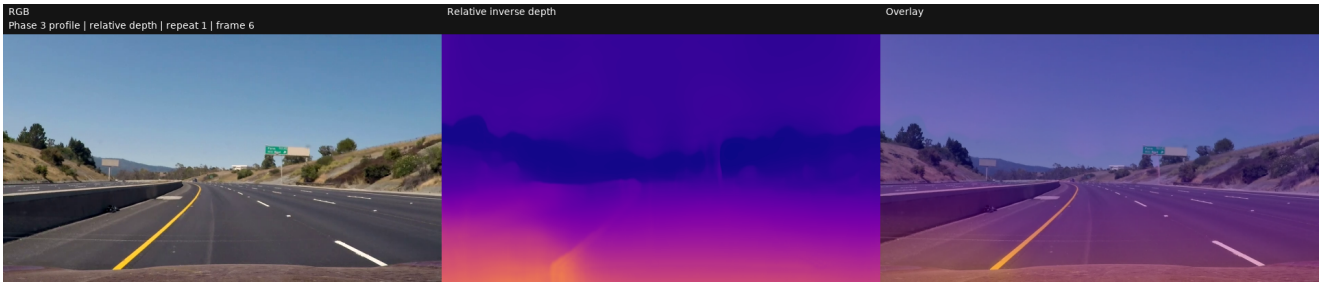
Module locations: src/model.py, src/preprocess.py, src/visualize.py, src/metrics.py and optional src/carla_capture.py. Phase 2 adds src/demo_data.py; Phase 3 adds src/profiling.py. project.py is the shared command-line entry point.

2. Working features and input preparation

Phase 1: reproducible dependency pins; official source/checkpoint acquisition and provenance checks; saved-image prediction; video/webcam input adapters; RGB/depth/overlay output; first-frame numerical arrays and preview; runtime logging; optional simulator capture/replay tools.

Phase 2: the prepare-demo command obtains a pinned Udacity driving sample and license. Every frame is decoded to check its count and dimensions, and a first-frame image fallback is saved. verify-demo validates the media, fallback image, baseline checkpoint and preprocessing identity after transfer to another laptop.

Frozen demo input	Recorded value
Source	Udacity / CarND-Advanced-Lane-Lines / project_video.mp4
Repository revision	91b080357290da37f533e2d0832f601c3e770d51
Video	1260 frames; 1280x720; 25 FPS; 50.4 source seconds
License	Repository MIT license retained as UDACITY_LICENSE.txt.
Ground truth	No aligned metric-depth ground truth is supplied by this sample.



Actual profile preview from the driving clip: RGB, relative inverse depth and overlay. Colours visualize model predictions; they do not establish measured accuracy.

Demo outputs: preview.png, inverse_depth.npy, relative_depth.npy, runtime.csv, summary.json and metrics.json. Without ground truth, metrics.json is empty and the summary explicitly marks accuracy unavailable.

Video replay is sequential. A processing cap below the native 25 FPS slows playback; it is not an interactive simulator or a guaranteed native-speed live stream.

3. Phase 3: stability and performance

Profile hardware: **NVIDIA GeForce GTX 1650**, Windows, Python 3.8.20, PyTorch 2.0.1+cu118. Input 640x192, batch one and two CPU threads. Each of 2 repeats excludes 5 full-pipeline warmup frames and measures the same 60 subsequent frames.

Numerical stability: every measured depth/inverse-depth pair is checked for shape, dtype, positivity, finiteness and reciprocal consistency. Both maps are hashed. All measured frame IDs and exact output fingerprints matched across the repeated runs on this machine.

Timing component	Median (ms)	P95 (ms)
Decode / RGB conversion / resize	5.75	8.07
Model only	22.40	28.50
Preprocessing + model + output transfer	30.37	38.63
Composite rendering	22.52	30.93
Numerical checks / hashing	2.18	2.74
Full measured headless cycle	61.30	78.42

Processing includes model time: these rows overlap and must not be added together. The measured cycle excludes asset verification, setup, warmup, display, pacing and output-file writes.

Repeat	Uncapped headless FPS	Measured frames
1	15.75	60
2	16.00	60

Rendering change: cache the palette, font and static labels; blend the overlay with installed OpenCV. A separate warmed 20-call rendering check measured median 55.06 ms before and 31.49 ms after (42.8% lower in this observation). Timing varies with system load; this is not a universal speedup guarantee.

The colour palette and numerical predictions are preserved. Overlay rounding may differ by one uint8 colour level. No model weights, preprocessing normalization or training configuration were changed.

4. Validation evidence and phase handoff

45 tests passed; failures: 0; errors: 0; skipped: 0. The regression suite includes the original Phase 1 and Phase 2 checks plus Phase 3 profiling and visualization checks.

Area	Evidence covered
Numerical / geometry	RGB order, tensor contract, depth decoding, intrinsics resizing, metric formulas and fixed colour scaling.
Safety / integration	Archive traversal/size checks, checkpoint hashes, frame synchronization, replay validation and cleanup paths.
Phase 2 transfer	Portable manifests, changed inputs/checkpoints, invalid metadata/paths, short decoding and source mismatch.
Phase 3 stability	Timing percentiles, warmup exclusion, finite reciprocal maps, repeat mismatch rejection and palette/overlay fidelity.
Actual model	Official-checkpoint repeatability and real driving-video CUDA profiling, separate from mocked tests.

Earlier displayed rehearsal: 350 real-video frames over 71.58 seconds at 4.89 FPS under a 5 FPS cap. This covers 14 seconds of source footage. It is a different workload from the uncapped headless Phase 3 profile; the two FPS values are not directly comparable.

Final-profile process peak working set: 631.95 MiB. PyTorch peak allocated GPU memory: 106.34 MiB. Working set is not total system RAM or committed memory; PyTorch allocations are not total VRAM usage.

Transition gate: keep the accepted baseline and input hashes unchanged, run the full suite, verify-demo, image inference and a short video smoke test before accepting a later phase. Write into new output directories. A failed optional experiment must not replace the working baseline.

Evidence locations (local, ignored): runs/phase3_tests.json; runs/phase3_profile_final/{summary.json,runtime.csv,fingerprints.json,preview.png}; runs/phase3_profile/summary.json (before rendering optimization); runs/phase3_render_before.json and runs/phase3_render_final.json.

5. Remaining work and reproducible usage

Requirement	Honest current position
Fine-tuning	Not performed. No prepared calibrated temporal training dataset or local pose/loss training pipeline; pretrained baseline retained.
Ground-truth accuracy	No full KITTI/CARLA evaluation completed. Tested formulas do not prove AbsRel/RMSE targets.
Live simulator / webcam	Native CARLA sensor validation and a physical-camera run remain pending.
Presentation laptop	RTX 3050 testing remains pending; current measurements belong to the local GTX 1650.
10+ FPS and metric scale	Uncapped headless FPS does not prove sustained displayed/live performance. Depth remains relative, without calibrated metre-scale output.
Vehicle control	Path planning, steering and throttle control are outside the original PDF scope and are not implemented.

Run from the project directory in PowerShell (choose unused output names):

```
$depthPython = '.\.venv\Scripts\python.exe'
& $depthPython project.py assets
& $depthPython project.py prepare-demo
& $depthPython project.py verify-demo
& $depthPython -m unittest discover -s tests -v
& $depthPython project.py profile --device cuda --frames 60 `
  --warmup 5 --repeats 2 --output runs/profile_review_01
& $depthPython project.py demo --source video `
  --video data/demo/driving.mp4 --manifest data/demo/input_manifest.json `
  --device cuda --frames 350 --fps 5 --output runs/demo_review_01
```

On the friend's laptop, transfer matching source, checkpoints, pinned third-party source and data/demo, then rebuild .venv. Verify CUDA, hashes and tests; rehearse the displayed demo there. Do not copy .venv or the CARLA runtime from the development machine.

References and provenance

Official Monodepth2 source and pretrained models

Udacity driving sample and repository license

Project repository

Original scope reference: monocular_depth_report_FINAL_FIXED (1).pdf, Sections 3.2-3.3 and Chapter 6. Current phase plans and repository code distinguish completed work from pending original targets.